CS560 - UI/UX Design & Implementation

9-1 Project Two: Usability Evaluation and Design Improvement

Malgorzata Debska

Southern New Hampshire University

Nathan Palmer

June 17, 2026

Usability Case Study: Evaluating GitHub's Repository Forking Workflow

User experience (UX) is a critical component of modern software development because it determines how efficiently and comfortably users can accomplish their goals. An application may have powerful features, but if users struggle to navigate the interface or complete common tasks, the overall value of the product decreases. Good UX design focuses on reducing frustration, improving accessibility, and creating workflows that feel natural and intuitive. According to Norman (2013), successful products are designed around human behavior and make it easy for users to understand what actions to take without unnecessary confusion.

For this case study, I selected GitHub and evaluated its repository forking workflow, which is one of the platform's most common collaborative development tasks. The workflow includes logging into GitHub, locating a public repository, creating a fork, cloning it to a local machine, creating a new branch, making code changes, committing those changes, and pushing them to a remote repository. This workflow is widely used by developers contributing to open-source projects or working on team-based software engineering efforts. The purpose of this evaluation is to determine how effectively GitHub supports users throughout this process by applying established usability testing methods, measuring workflow efficiency, analyzing design choices, evaluating the interface using usability heuristics, and providing recommendations for improvement. These techniques allow designers and software engineers to make evidence-based decisions that enhance productivity and overall user satisfaction (Lazar et al., 2017).

Usability Testing and Evaluation Methods

A comprehensive usability evaluation should use multiple methods because each approach provides different insights into how users interact with an interface. For this case study, I would combine heuristic evaluation, task-based usability testing, think-aloud protocols, and cognitive walkthroughs. The primary evaluation method would be a heuristic evaluation, which uses Nielsen's Ten Usability Heuristics to identify interface strengths and weaknesses. Nielsen (1994) explains that heuristic evaluations are effective because they provide a structured framework for assessing visibility of system status, consistency, error prevention, recognition rather than recall, user control, and other key usability principles. This method allows evaluators to systematically review GitHub's interface and identify areas that may confuse users or reduce efficiency. The second method would be task-based usability testing. Participants would be asked to complete the repository forking workflow while observers record how successfully they complete each step. Measuring actual user performance provides valuable evidence about whether the interface supports users in realistic scenarios. During testing, participants could be instructed to fork a repository, create a branch, make a small code modification, and push their changes. I would also incorporate a think-aloud protocol, where participants explain their thoughts while interacting with the system. This approach often reveals misunderstandings that are difficult to identify through observation alone. For example, a user may hesitate before selecting the "Fork" button because they are unsure whether it creates a copy or modifies the original project. I would perform a cognitive walkthrough to evaluate how first-time users experience the workflow. This method focuses on whether users can naturally determine the next step based solely on the interface design. Lazar et al. (2017) emphasize that combining expert reviews with observations

of real users provides a richer understanding of usability problems and supports more effective design improvements.

To strengthen this evaluation, the usability study should include 8 –10 participants with diverse levels of Git and GitHub experience, ranging from beginners to advanced developers. Each participant would be asked to complete the repository forking workflow independently while observers record task completion times, navigation patterns, and any errors encountered. During the think-aloud sessions, participants would explain their decisions and any confusion they experience while interacting with the interface. Collecting both quantitative performance data and qualitative user feedback would provide a more comprehensive understanding of GitHub's usability and identify issues that may not be apparent through expert review alone. Accessibility should also be incorporated into the evaluation process. In addition to measuring workflow performance, the interface should be assessed for keyboard navigation, screen reader compatibility, color contrast, and the clarity of labels and controls. Considering accessibility ensures that users with diverse abilities can effectively complete tasks and align with user-centered design principles.

Metrics and Approaches Used to Measure Usability and Workflow Efficiency

Evaluating usability requires measurable criteria that provide objective evidence about user performance. One of the most important metrics is the task completion rate, which measures whether participants can successfully complete the repository forking workflow without assistance. If many users fail to finish the task, the interface may contain barriers requiring

redesign. Another important metric is time spent on tasks. Measuring how long users take to complete the workflow provides insight into efficiency and highlights steps that create delays. If users consistently spend excessive time locating buttons or interpreting instructions, this suggests opportunities for simplification.

Error frequency is also valuable because it identifies where users encounter problems. In GitHub's workflow, common mistakes might include committing code to the wrong branch, cloning an incorrect repository, or misunderstanding synchronization between local and remote repositories. Recording these errors allows evaluators to identify patterns and prioritize improvements. The evaluation would also examine the number of clicks and navigation steps required to complete tasks. Unnecessary interactions increase cognitive load and reduce productivity. Streamlined workflows produce a more satisfying user experience.

User satisfaction would be measured using the System Usability Scale (SUS), a standardized assessment tool that has been widely adopted across many software applications. Brooke (1996) explains that the SUS provides a reliable method for measuring perceived usability and comparing user experiences across different systems. Finally, learnability would be evaluated by comparing first-time users with experienced developers. If beginners struggle significantly more than experienced users, the interface may require additional guidance or onboarding materials. Together, these metrics provide a balanced assessment of effectiveness, efficiency, and satisfaction.

Beyond task completion rate, time on task, and error frequency, several additional metrics would provide valuable insight into user performance. The number of requests for assistance during testing can indicate areas where users struggle independently. Navigation success rate can reveal

whether participants consistently locate important controls without unnecessary searching. User confidence ratings collected after task completion can help determine whether participants feel comfortable repeating the workflow on their own, while measuring error recovery time demonstrates how easily users can recognize and correct mistakes. Together, these metrics create a more complete picture of workflow efficiency, effectiveness, and overall user satisfaction.

Analysis of Design Choices and Their Impact on User Workflows

GitHub's interface demonstrates several thoughtful design decisions that contribute to workflow efficiency, particularly for experienced developers. Repository pages follow a consistent structure, with navigation tabs for source code, issues, pull requests, actions, and settings located in predictable positions. This consistency allows returning users to locate information quickly and reduces unnecessary searching. The overall layout is clean and organized, making it easier to focus on development tasks without excessive visual clutter. Information is grouped logically, and commonly used actions remain visible throughout the workflow. Norman (2013) notes that interfaces should minimize cognitive effort by presenting information in familiar and predictable ways, which GitHub accomplishes well. Despite these strengths, some design choices create challenges for less experienced users. The "Fork" button is clearly visible but provides little explanation about its purpose. Someone unfamiliar with version control concepts may not immediately understand that forking creates a personal copy of a repository rather than modifying the original project.

Navigation between GitHub's browser interface and local development tools also introduces complexity. After creating a fork, users must switch to terminal commands or desktop applications to clone repositories and manage branches. This interruption requires users to

remember multiple commands and concepts, increasing cognitive load, and creating opportunities for mistakes. The platform's reliance on specialized terminology such as "commit," "merge," "branch," and "rebase" may also hinder usability for beginners. Although these terms are standard within software engineering, they require prior knowledge that inexperienced users may not possess. Also, contextual explanations or guided tutorials could reduce confusion and improve accessibility. GitHub's design strongly supports experienced developers but presents a steeper learning curve for users with limited background in distributed version control systems.

Evaluation Using Usability Heuristics

Applying Nielsen's usability heuristics provides a structured way to evaluate GitHub's interface. The platform performs very well in terms of visibility of system status by providing immediate confirmation after actions such as commits, pushes, merges, and pull request creation. Users receive clear feedback that their actions have been completed successfully (Nielsen, 1994). Regarding the match between the system and the real world, GitHub reflects terminology commonly used in software engineering. However, this specialized vocabulary may be difficult for novice users who are unfamiliar with version control concepts.

GitHub also demonstrates strong consistency and standards. Navigation menus, icons, layouts, and workflows remain uniform across repositories, allowing experienced users to transfer knowledge from one project to another with minimal adjustment. The platform supports user control and freedom by allowing developers to switch branches, amend commits, and reverse many actions before publication. Nevertheless, correcting mistakes involving remote repositories may still require advanced technical knowledge. In terms of error prevention, GitHub includes

confirmation dialogs before destructive actions such as deleting repositories or force-pushing changes. These safeguards reduce the likelihood of accidental data loss.

The heuristic of recognition rather than recall is partially satisfied because navigation tabs and visible controls help users locate features. However, command-line interactions still require memorization of Git commands, increasing reliance on recall. GitHub excels in flexibility and efficiency of use, offering shortcuts, automation features, command-line integration, and extensive customization options that significantly improve productivity for advanced users.

The interface also follows principles of aesthetic and minimalist design, presenting information in a clean, organized format without excessive decoration. Error messages frequently include explanations and links to documentation, supporting users in recognizing and recovering problems. GitHub provides extensive help and documentation, including tutorials, guides, and community resources that assist users in learning more advanced workflows (Nielsen, 1994).

Although GitHub performs well across many of Nielsen's heuristics, several opportunities for improvement remain. For example, the principle of recognition rather than recall is challenged when users leave the web interface and must remember Git command-line syntax. Likewise, the heuristic of matching the system to the real world is only partially satisfied because terminology such as "fork," "rebase," and "upstream" may be unfamiliar to novice users. While GitHub offers documentation to address these concepts, integrating clearer explanations directly into the interface could reduce cognitive load and improve first-time user success rates.

Recommendations for Improving Workflow Efficiency and Usability

Based on this evaluation, several improvements could make GitHub more accessible while preserving its powerful capabilities. First, GitHub should implement an interactive onboarding tutorial for new users. A guided experience explaining repositories, branches, commits, forks, and pull requests would reduce confusion and improve learnability. Second, context-sensitive tooltips could provide simple explanations when users hover over technical terminology. Short descriptions written in plain language would make specialized concepts easier to understand. Third, GitHub could introduce a guided workflow assistant that walks users through forking, cloning, branching, committing, and pushing changes step by step. Such assistance would reduce errors and improve task completion rates for beginners. Fourth, stronger visual indicators distinguishing local and remote repositories would help users understand where their code changes are stored and synchronized. This enhancement could reduce common mistakes involving version control.

In addition to onboarding tutorials and contextual guidance, GitHub could improve workflow efficiency by incorporating interactive visual diagrams that illustrate the relationships between repositories, forks, branches, and pull requests. These visual aids would help users understand complex version control concepts more quickly. The platform could also introduce a beginner-friendly interface mode that simplifies terminology and highlights only the most used features until users gain experience. AI-powered contextual assistance capable of explaining technical concepts in plain language could further reduce confusion and support independent learning. Finally, implementing automated workflow validation that warns users before pushing an unintended branch or repository could prevent common mistakes and improve overall task success rates.

Limitations of the Evaluation

One limitation of this evaluation is that it is based on expert analysis and proposed usability testing methods rather than results collected from a live user study. Future research involving a larger and more diverse participant sample would strengthen the findings by providing empirical evidence of user behavior across unique experience levels. Nevertheless, the combination of heuristic evaluation, usability metrics, and workflow analysis offers a solid foundation for identifying meaningful improvements.

Conclusion

This case study demonstrates that GitHub provides a highly capable platform for collaborative software development while also presenting opportunities for usability improvements. The repository forking workflow benefits from consistent navigation, clear system feedback, flexible tools, and extensive documentation, making it highly effective for experienced users.

However, the evaluation also reveals challenges for beginners, particularly regarding technical terminology, transitions between web and local environments, and understanding distributed version control concepts. Applying heuristic evaluation, usability testing, and measurable performance metrics provides valuable evidence for identifying these issues and developing targeted solutions. As software engineering continues to emphasize user-centered design, evaluating workflows through established UX principles becomes increasingly important. Incorporating guided onboarding, contextual assistance, and simplified workflows, GitHub could further enhance accessibility without sacrificing the advanced functionality that makes it a leading collaboration platform. Creating software that aligns with users' expectations and

supports natural interactions is a key principle of user-centered design and contributes to long-term usability and satisfaction (Norman, 2013).

References:

Brooke, J. (1996). *SUS: A "quick and dirty" usability scale*. In P. W. Jordan, B. Thomas, I. L. McClelland, & B. Weerdmeester (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Lazar, J., Feng, J. H., & Hochheiser, H. (2017). *Research methods in human-computer interaction* (2nd ed.). Morgan Kaufmann.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Norman, D. A. (2013). *The design of everyday things* (Revised and expanded ed.). Basic Books.